

Digital Ecosystems Architecture

A Three-Baseline Framework for Coherent Digital Realization

Authors: James D. Longmire

Micah Longmire Sr. AI Architect

ORCID: 0009-0006-7608-9322

Working Paper, Conceptual Base Layer

Epistemic status: *Foundational framing. Intended to stand alone as a general architecture frame and to serve as the substrate for a forthcoming AI-specific extension (AIDE-AF).*

Summary

Modern enterprises do not fail their architecture by lacking architects. They fail it by losing coherence across the disciplines architects practice. Capability maps are drawn that never constrain system design. Systems are built that trace to no business outcome. Solutions are delivered that satisfy a local requirement while degrading the enterprise that has to operate them afterward. Each discipline does competent work; the work does not add up.

Digital Ecosystems Architecture addresses that failure directly. It organizes enterprise architecture around three baselines, each owned by a distinct architecture discipline, each answering one governing question, and binds them together with traceability that runs in both directions. The three are the Digital Capability Baseline, the Digital Technical Baseline, and the Digital Operational Baseline. The qualifier is deliberate: it marks the scope, addressed in Section 8, to capabilities whose means of realization is substantially technical.

The Digital Capability Baseline, owned by Enterprise Architecture, answers what the enterprise must be able to do. The Digital Technical Baseline, owned by Systems Architecture, answers how that capability is technically realized. The Digital Operational Baseline, owned by Solutions Architecture, answers how the resulting solution is delivered, operated, governed, and sustained. Traceability connects them: from business outcome to capability, capability to system, system to solution, solution to operation, and operation back to measured performance that revises the baselines above it.

The framework is deliberately scoped to digitally-realized capabilities: capabilities whose means of realization is substantially technical. That scope is stated rather than assumed, for reasons the closing section makes explicit.

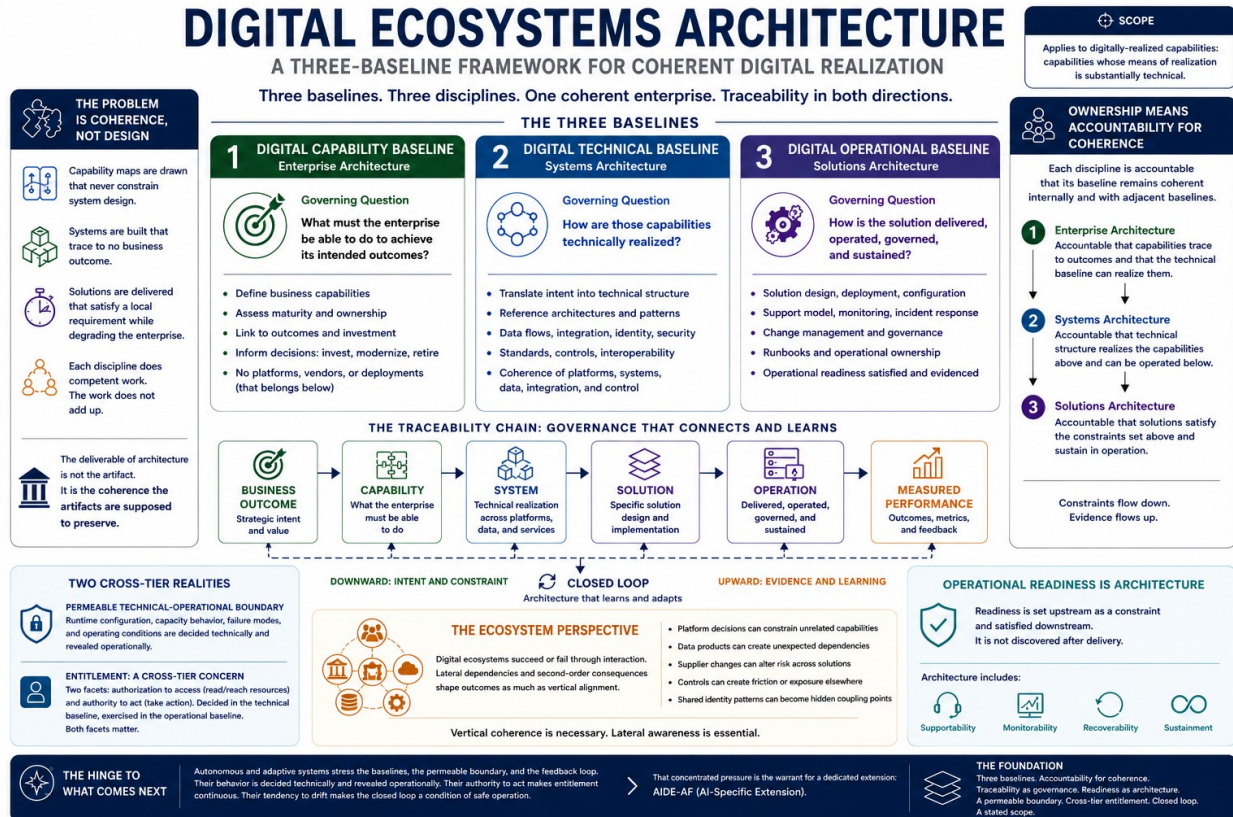


Figure 1. Digital Ecosystems Architecture at a glance. This is an initial working model of the conceptual base layer; its application should be tailored to each organization's requirements. The ecosystem (lateral dependency) dimension is directional and still under development.

1. The Problem Is Coherence, Not Design

Enterprise technology environments are structurally complex, and the instinct is to treat complexity as a design problem: better models, cleaner diagrams, more rigorous specifications. That instinct is not wrong, but it locates the difficulty in the wrong place. A single business capability now depends on multiple platforms, internal and third-party services, identity and data infrastructure, security controls, operational support, and increasingly on systems that act with some degree of autonomy. Designing any one of these well is achievable. Keeping all of them coherent with one another, and with the strategic intent they are supposed to serve, is the harder and more neglected problem.

Coherence fails in recognizable ways. A capability appears mature on a capability map while the systems beneath it remain brittle. A platform is technically sound but poorly aligned to any outcome the enterprise actually pursues. A solution is delivered on time and bypasses the standards that would have let it integrate, be governed, or be supported. None of these is a design failure in the narrow sense. Each component may be well designed. The failure is in the joins.

Most architecture practice produces artifacts. Capability models, reference architectures, solution designs. Artifacts are necessary, but they are not the same as coherence, and an enterprise can accumulate a great many of them while becoming less coherent rather than more. The deliverable of architecture is not the artifact. It is the coherence the artifacts, taken together, are supposed to

preserve. Digital Ecosystems Architecture treats that coherence as the primary object and the baselines as the means of maintaining it.

2. Three Baselines, Three Disciplines, Three Questions

The framework rests on three baselines. Each corresponds to an established architecture discipline and is defined by a single governing question.

The Digital Capability Baseline is owned by Enterprise Architecture. Its governing question is: *what must the enterprise be able to do to achieve its intended outcomes?* Its concern is strategic and business alignment. It defines business capabilities, their maturity, their ownership, and their relationship to outcomes and investment. It is not a static taxonomy. A capability baseline that does not inform decisions about where to invest, modernize, or retire has become decoration. It does not specify platforms, vendors, or deployment models; those belong to the tier below.

The Digital Technical Baseline is owned by Systems Architecture. Its governing question is: *how are those capabilities technically realized?* Its concern is the coherence of platforms, systems, data, integration, and control. It translates capability intent into technical structure: the reference architectures, integration patterns, data flows, identity and security boundaries, and standards that determine how a capability is enabled, constrained, secured, and scaled. This baseline is where the enterprise moves from intent to structure.

Systems Architecture is used here in the broad enterprise sense: the architecture of technical realization across platforms, systems, data, integrations, controls, and interoperability patterns. It is not limited to the design of a single system. The narrower usage, in which systems architecture means the internal design of one application, product, or platform, is not the primary sense intended here. In this framework that narrower design work is treated as part of solution realization, unless it establishes reusable technical structure across capabilities, in which case it rises to the Digital Technical Baseline.

The Digital Operational Baseline is owned by Solutions Architecture. Its governing question is: *how is the solution delivered, operated, governed, and sustained?* Its concern is the integrity of delivery and sustainment. It defines the specific solution design, deployment, configuration, support model, monitoring, incident response, change management, and the runbooks and ownership that make a system operable after it ships. This baseline is where architecture becomes operationally real, and where the consequences of incoherence in the baselines above it become visible.

The correspondence is intentionally clean: one discipline, one baseline, one question. But the cleanness conceals a question about ownership that has to be answered before the framework holds together, and the next section answers it.

3. Ownership Means Accountability for Coherence

A naive reading of the three-baseline structure assigns each discipline exclusive authorship of its baseline. Enterprise Architecture writes the Digital Capability Baseline, Systems Architecture writes the Digital Technical Baseline, Solutions Architecture writes the Digital Operational Baseline, and each hands its output down the line. That reading is tidy and wrong, and two of the framework's own commitments expose why.

The framework holds that operational readiness is an architectural concern, set upstream and satisfied downstream rather than discovered after delivery (Section 5). It also holds that traceability is itself a

governance mechanism rather than documentation (Section 4). If both are true, no single discipline can author its baseline in isolation. The criteria a solution must satisfy to be operationally ready, including supportability, monitorability, recoverability, and sustainment, are shaped by decisions made in the Digital Capability and Digital Technical Baselines. The Digital Operational Baseline is therefore not authored solely by Solutions Architecture; it is constrained from above and satisfied below.

The resolution is to define ownership as accountability for coherence rather than exclusive authorship. To own a baseline is to be accountable that it remains coherent, internally and with the baselines adjacent to it. Each discipline sets constraints the next must satisfy and returns evidence the previous must account for. Enterprise Architecture is accountable that capabilities trace to outcomes and that the technical baseline can realize them. Systems Architecture is accountable that technical structure realizes the capabilities above and can be operated below. Solutions Architecture is accountable that solutions satisfy the constraints set above and sustain in operation.

This distinction is not a verbal nicety. It determines how the framework behaves at its boundaries, and it is the reason the three tiers can be distinct in concern while remaining coupled in practice.

4. Traceability Is Governance

Traceability in most architecture practice is treated as documentation: a record, assembled after decisions are made, showing that a system connects to a requirement that connects to a goal. Documentation traceability is constructed retrospectively and consulted rarely. It is the artifact, not the discipline.

The framework makes a stronger claim. Traceability is the mechanism by which architectural governance actually operates. If an initiative cannot be traced from business outcome through capability, system, and solution to operation and measured performance, then governance has nothing to govern with. It is left to approve or reject on assertion, on the persuasiveness of whoever is presenting, rather than on whether the initiative coheres with the enterprise it joins. An initiative that cannot be traced is not merely undocumented. It is ungoverned, because the governance function has no basis on which to decide.

This reframes what traceability is for. It is not a record of decisions; it is the precondition for making them well. A governance gate that asks “does this trace?” is asking the only question that distinguishes architectural decision from procurement decision. When the trace is intact, governance can reason about coherence. When it is broken, governance is operating blind regardless of how many artifacts are in the binder.

Traceability also runs in both directions, which most treatments understate. The downward chain expresses intent: outcome shapes capability, capability shapes system, system shapes solution. The upward chain expresses reality: measured performance revises the Digital Operational Baseline, which revises the Digital Technical Baseline, which revises the Digital Capability Baseline. A traceability model that runs only downward is a plan. A model that closes the loop is an architecture that learns. The closing of that loop is treated here as intrinsic rather than optional, for reasons that become decisive in the systems the extension will address.

5. Operational Readiness Is an Architectural Concern

The convention in much enterprise architecture practice is that architecture ends at design and implementation governance, and that operations begins afterward. Runbooks, support boundaries, monitoring, incident response, and sustainment are treated as the downstream concern of operations teams, picked up once the architects have moved on. This convention is the source of a recurring and expensive failure: solutions that are architecturally sound and operationally unviable.

The framework rejects the convention. Operational readiness is set as a constraint in the upper baselines and satisfied in the Digital Operational Baseline; it is not discovered after delivery. Whether a capability can be monitored, supported, recovered, and sustained is decided, often implicitly, when the capability is scoped and when its technical realization is chosen. By the time a solution reaches operations, the readiness question has already been answered; operations merely learns what answer was chosen for it. To treat readiness as post-delivery work is to make an architectural decision and decline to be accountable for it.

Placing operational readiness inside architecture is the framework's most consequential integration, because it joins a tradition that enterprise architecture frameworks have historically held at arm's length. The Digital Capability and Digital Technical Baselines draw on the established vocabulary of enterprise and systems architecture. The Digital Operational Baseline draws on the service-management and operational-reliability traditions, runbooks, monitoring thresholds, incident response, sustainment, ownership, that those frameworks tend to treat as outside their scope. The join is the point. An architecture that stops at design has declined responsibility for whether its designs survive contact with operation.

6. The Traceability Chain and Its Boundaries

The framework's structure can be stated as a single chain:

business outcome to capability to system to solution to operation to measured performance, and back.

Each link is a relationship of constraint in the downward direction and evidence in the upward. The chain is the spine of the framework, and two of its features deserve specific attention because they determine how the framework behaves under stress.

The boundary between the Digital Technical and Digital Operational Baselines is the most permeable of the three. The boundary between the Digital Capability and Digital Technical Baselines is relatively clean: intent and realization are genuinely different kinds of statement. The boundary between the Digital Technical and Digital Operational Baselines is not clean, and pretending otherwise sets up a failure. Runtime configuration, capacity behavior, failure modes, and the actual conditions under which a system operates are simultaneously matters of technical design and operational reality. A system's behavior under load is decided in the Digital Technical Baseline and revealed in the Digital Operational Baseline. This permeability is not a defect to be engineered away. It is a property of the boundary, true of complex systems generally, and a framework that draws the line too crisply will be broken by the first system whose runtime behavior refuses to respect it.

Entitlement is a cross-tier concern with two facets. Who or what is permitted to do what, against which resources, under whose accountability, is not the property of any single baseline. It is decided in the Digital Technical Baseline, where access models, authorization boundaries, and identity structures are

designed, and it is exercised in the Digital Operational Baseline, where those permissions act under live conditions. Entitlement has two facets that different domains weight differently: authorization to access, the right to read or reach a resource, and authority to act, the right to take action against it. A data-centric domain leans on the access facet; a domain of autonomous or acting systems leans on the authority facet. Naming entitlement broadly enough to hold both facets, and locating it across rather than within tiers, keeps it general. A framework that defined entitlement narrowly, as access alone or as authority alone, would have quietly shaped itself to one domain.

These two features, the permeable boundary and the two-faceted cross-tier entitlement object, are stated here as general properties of digitally-realized architecture. They are also, not coincidentally, the features that the framework's intended extension will rely on most heavily.

7. A Note on Ecosystem

The framework is named for ecosystems because digitally-realized capabilities are not merely delivered through vertical chains of intent, realization, and operation. They are delivered through interacting components: platforms, data products, services, suppliers, controls, teams, users, automation, and operating processes that affect one another laterally as well as hierarchically.

The three-baseline model gives the framework its spine. It explains how intent moves downward and evidence moves upward. But digital ecosystems also fail sideways. A platform decision can constrain an unrelated capability. A data product can become a dependency for systems its original sponsors never considered. A supplier change can alter the operational risk of multiple solutions at once. A control introduced for one domain can become friction or exposure in another. A shared identity pattern can become the hidden coupling point between otherwise unrelated services. These are ecological failures because they arise from interaction, dependency, and feedback across the environment, not simply from a broken vertical trace.

For that reason the term ecosystem is not decorative. It names the architectural condition the framework is meant to govern: a digitally-realized enterprise in which capabilities, systems, solutions, controls, suppliers, and operators form a shared environment. The traceability chain preserves vertical coherence. The ecosystem frame keeps lateral dependency and second-order consequence in view.

This base layer does not yet exhaust the ecological dimension. It establishes the minimum structure required to govern it: baselines, ownership, traceability, operational readiness, and feedback. Later extensions can develop the lateral dependency model more fully without changing the foundation.

8. Scope, and the Edge of the Framework

The framework applies to digitally-realized capabilities, those whose means of realization is substantially technical. This scope is a finding, not a convenience.

A capability realized chiefly through people, doctrine, training, and procedure, with technology as incidental support, strains the framework's spine, and the naming is where the strain shows. A capability baseline in the general sense accommodates such a case; capability-based planning came from exactly such domains. A general operational baseline accommodates it. But the technical baseline under-describes it, because for such a capability the means of realization is mostly not technical, and the middle baseline is named for a mode of realization that does not dominate. The technical baseline is doing two jobs that coincide when realization is technical and separate when it is not: it names the

means of realization, and it serves as the middle layer between strategic intent and operational delivery. When realization is mostly human, those jobs come apart and the framing weakens.

This is precisely what the qualifier in the Digital Technical Baseline names and excludes. The three baselines are Digital Capability, Digital Technical, and Digital Operational not as decoration but because the digital qualifier is the scope boundary made structural. Each tier carries the limit in its name. A human-realized capability is out of scope not by exception but by definition: its realization is not digital, so it does not fall under a Digital Technical Baseline, and the framework does not claim it.

The correct response is to state the boundary rather than stretch the framework to cover what it does not fit. The framework is for digitally-realized capabilities. The word *digital* in its name is doing real work, marking the domain rather than decorating it. A framework that knows its edge is more credible than one that claims none, and locating the edge honestly is more useful than asserting a generality that the first hard case would expose.

9. The Hinge to What Comes Next

This base layer is built to stand on its own as a general framework for digitally-realized architecture, and to be extended. Where the framing had a genuine choice between two equally general options, it took the one that is also the more truthful: traceability that closes the loop rather than terminating, a technical-operational boundary acknowledged as permeable rather than drawn crisply, entitlement elevated to a named cross-tier object with both its facets. In each case the choice is defensible without reference to any extension. The base layer should read as a more precise account of the conventional stack, not as a framework quietly tailored to one future extension.

The forward claim is narrow and is the only one the base layer makes about its successors. Emerging classes of systems, autonomous and adaptive ones in particular, stress these baselines, this boundary, and this feedback loop harder than prior technology did. Their behavior is decided technically and revealed operationally, collapsing the technical-operational boundary further. Their authority to act makes entitlement a live and continuous concern rather than a settled one. Their tendency to drift makes the closed feedback loop a condition of safe operation rather than a refinement. That concentrated pressure is the warrant for a dedicated extension, and it is where the next layer of this work begins.

The base layer ends here. It is intentionally small: three baselines (Digital Capability, Digital Technical, Digital Operational), ownership as accountability for coherence, traceability as governance, operational readiness as architecture, a permeable boundary, a cross-tier entitlement object, a closed loop, and a stated scope. A foundation should be small enough to extend in more than one direction. Whether the first extension is the AI-specific one or another, the substrate beneath it is the same.